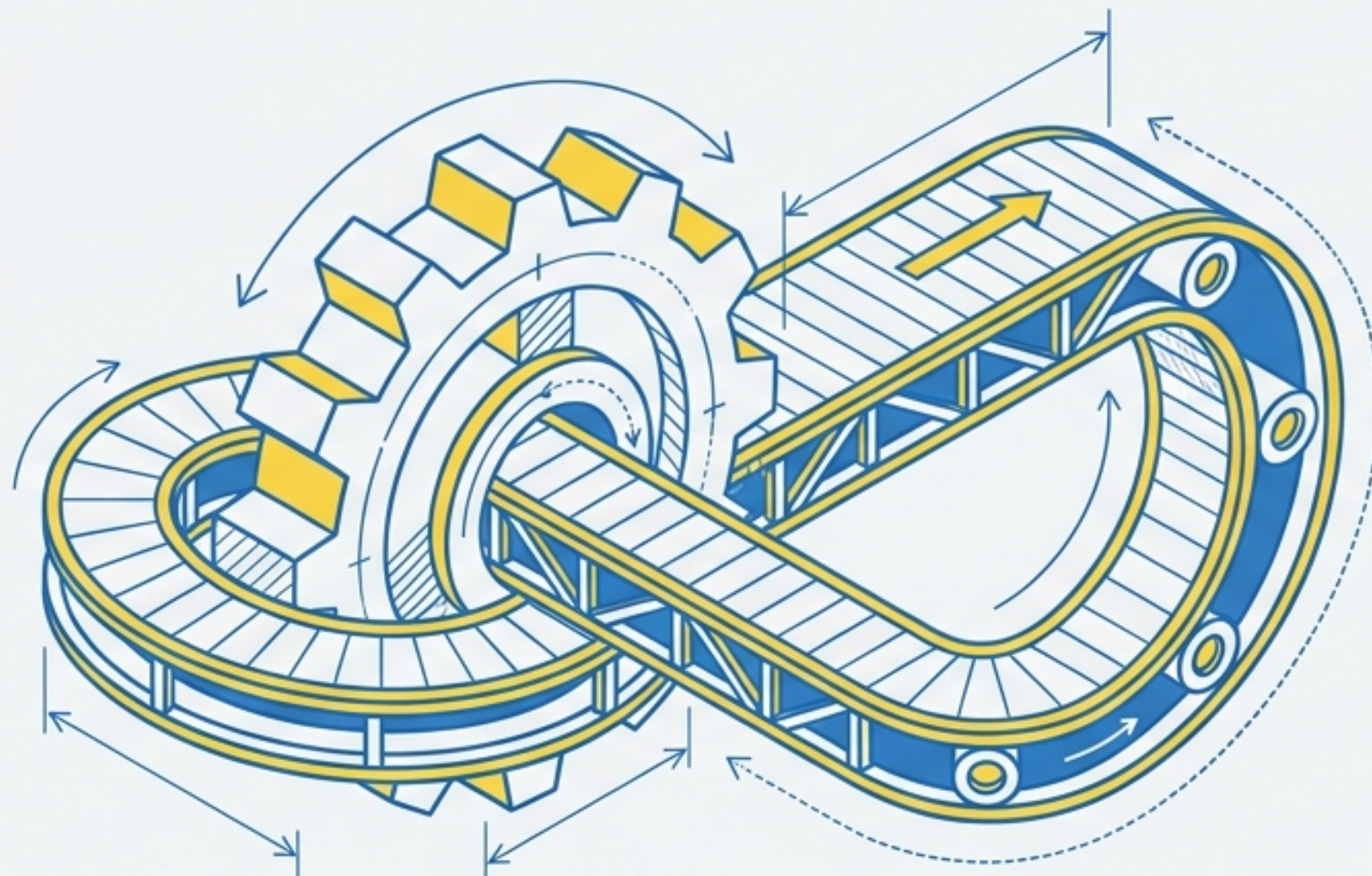
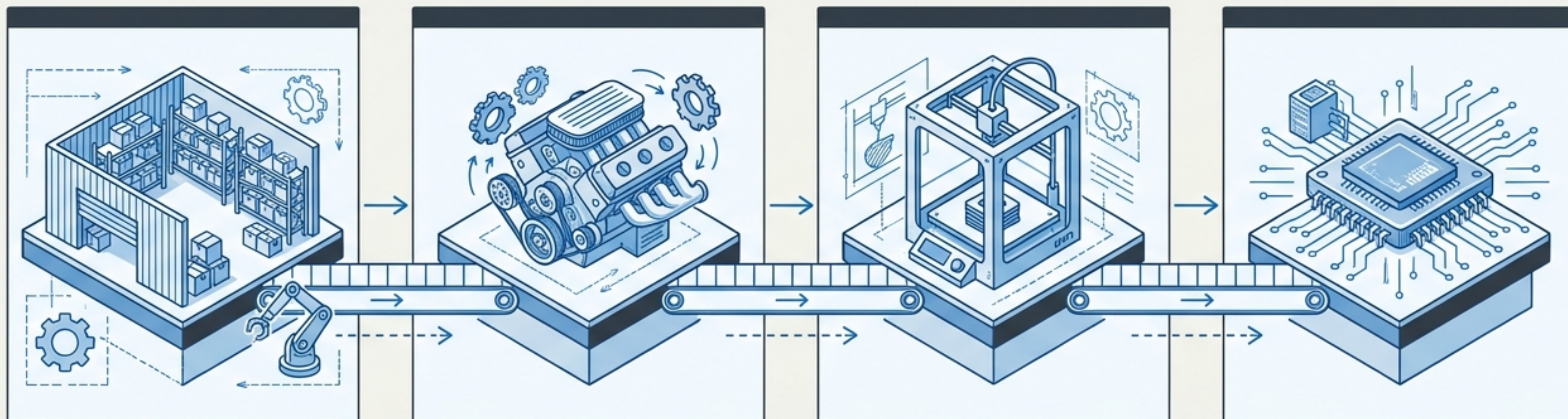




O Ecossistema de Iteração em Python

Da Coleção Estática ao Consumo Dinâmico de Dados

Sumário: A Linha de Montagem de Dados



Estação 1

Coleções e Iteráveis
(O **Armazém**)



Estação 2

Iteradores
(O **Motor**)

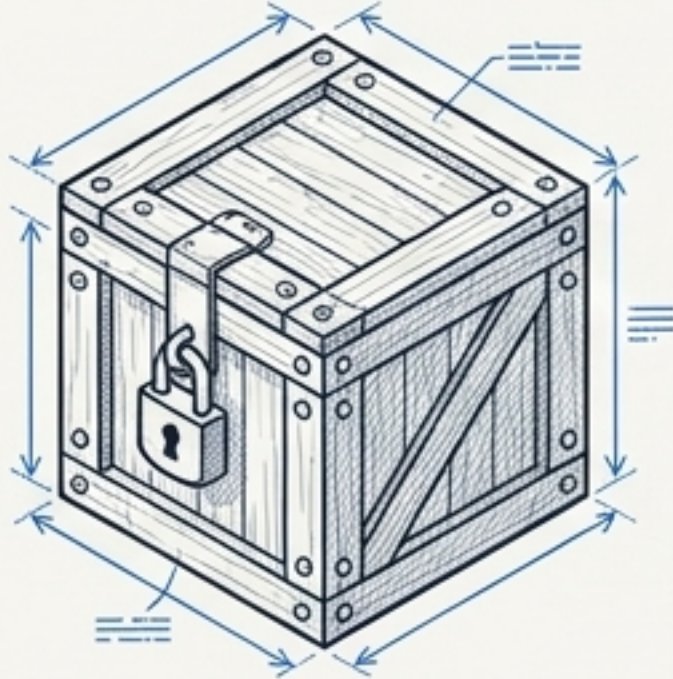
Estação 3

Geradores
(A Impressora 3D)

Estação 4

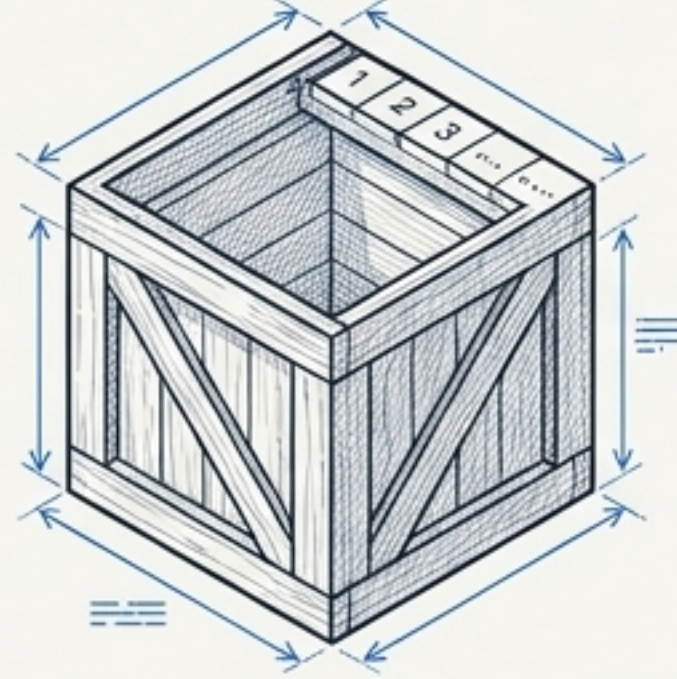
Corrotinas
(O Processador)

Coleções: O Armazém de Dados



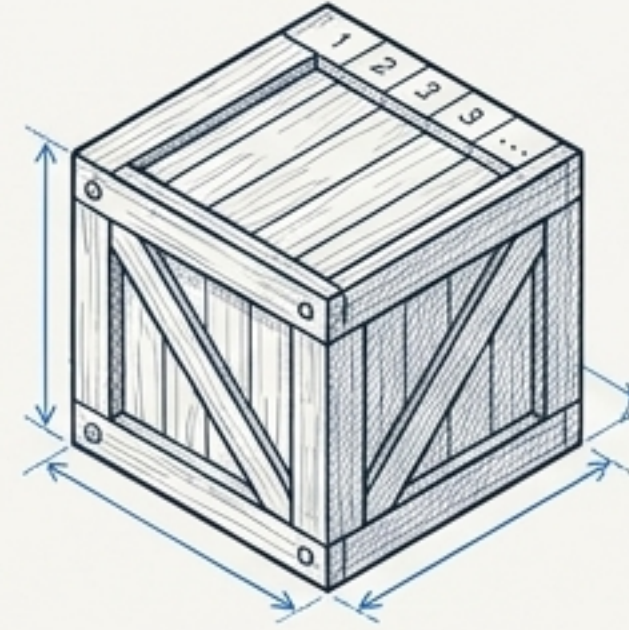
Tuplas

Ordenadas, Imutáveis



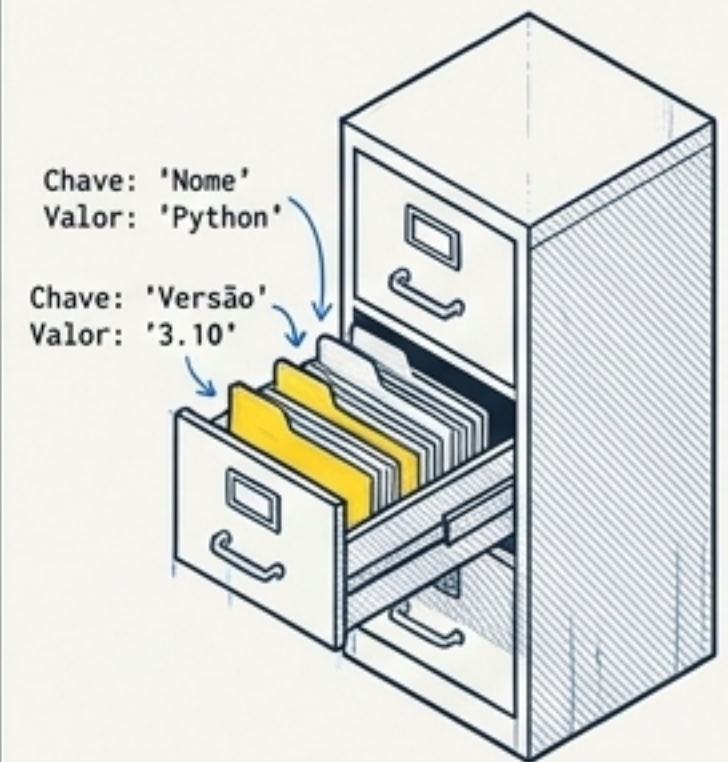
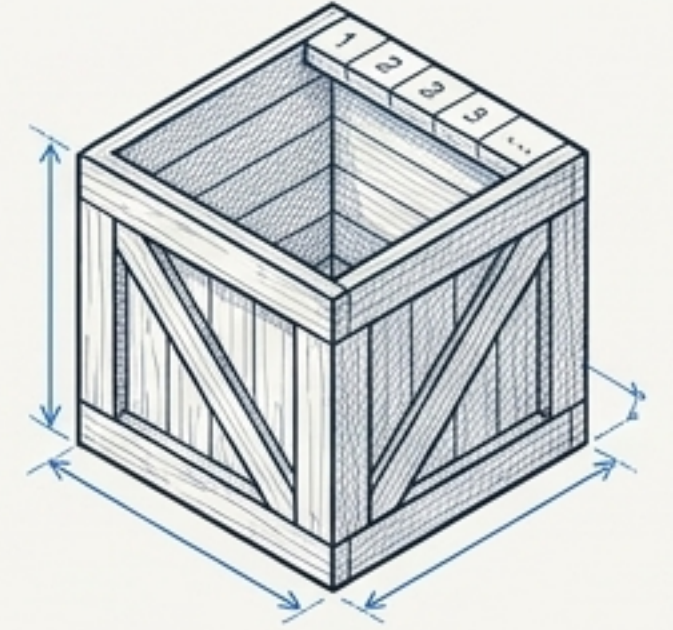
Listas

Ordenadas, Mutáveis



Sets

Não-ordenados, Sem Duplicatas



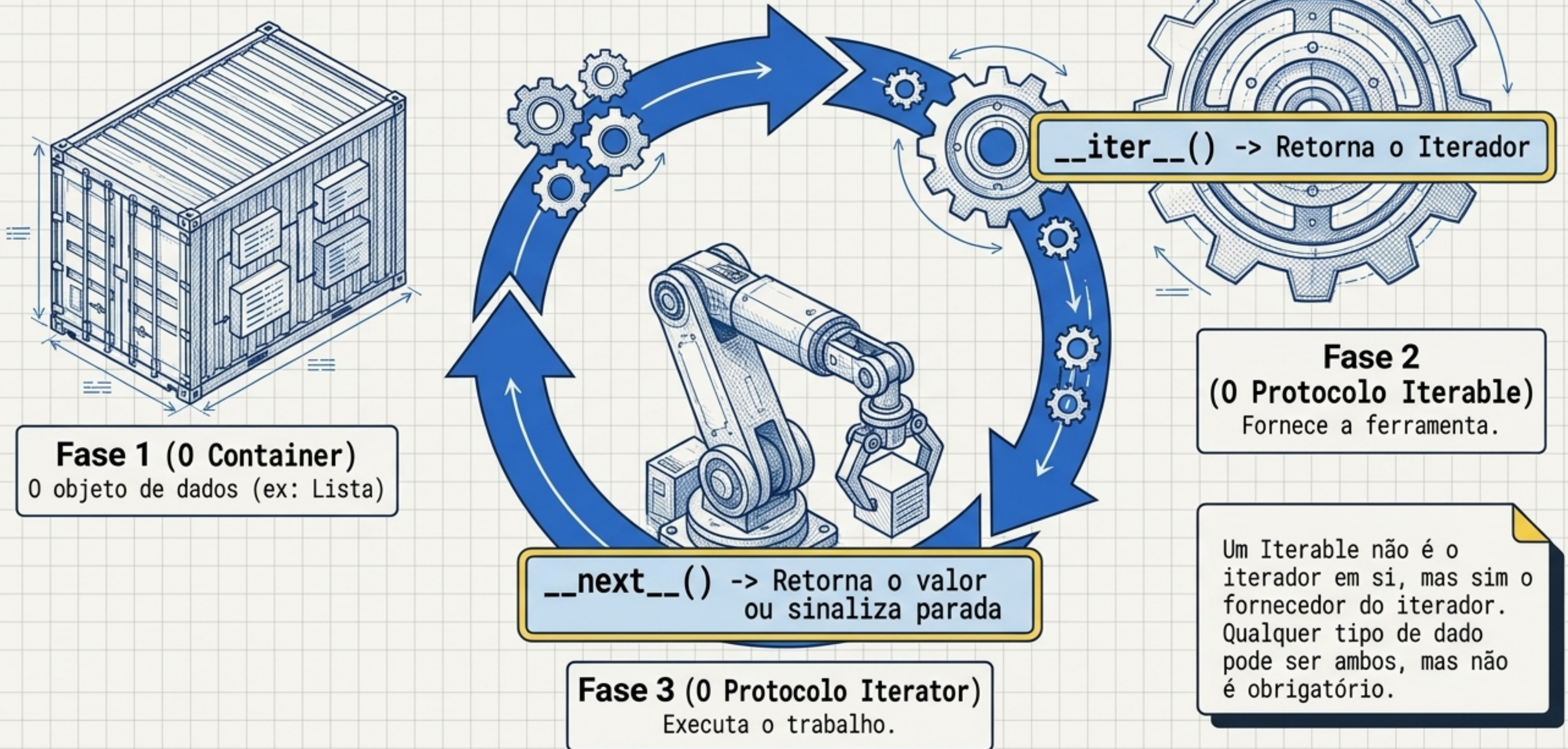
Chave: 'Nome'
Valor: 'Python'

Chave: 'Versão'
Valor: '3.10'

Dicionários

Chave-Valor, Indexados

O Paradigma da Iteração



Iterables: O Fornecedor

```
for item in container:  
    print(item)
```

O loop `for` espera receber um Iterable.

O método mágico `__iter__()` deve obrigatoriamente fornecer uma referência para um objeto Iterator.

Iterators: O Motor de Busca

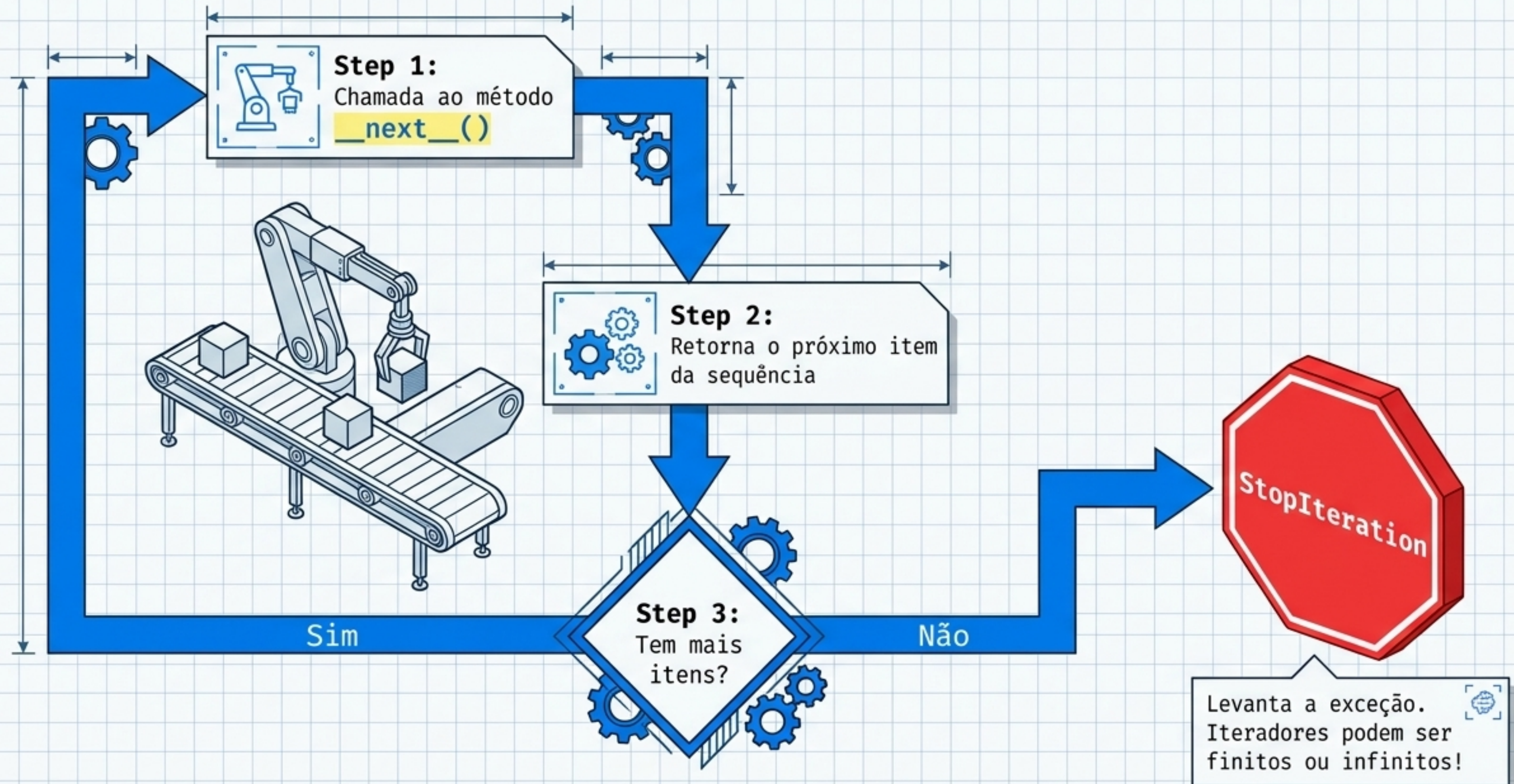


Ilustração Guiada: A Classe de Iteração Pura

```
class Evens(object):  
    def __init__(self, limit):  
        self.limit = limit  
        self.val = 0  
  
    def __iter__(self):  
        return self  
  
    def __next__(self):  
        if self.val > self.limit:  
            raise StopIteration  
        else:  
            return_val = self.val  
            self.val += 2  
            return return_val
```

Zona 1: Estado

Mantém o estado atual (self.val = 0).

Zona 2: O Fornecedor

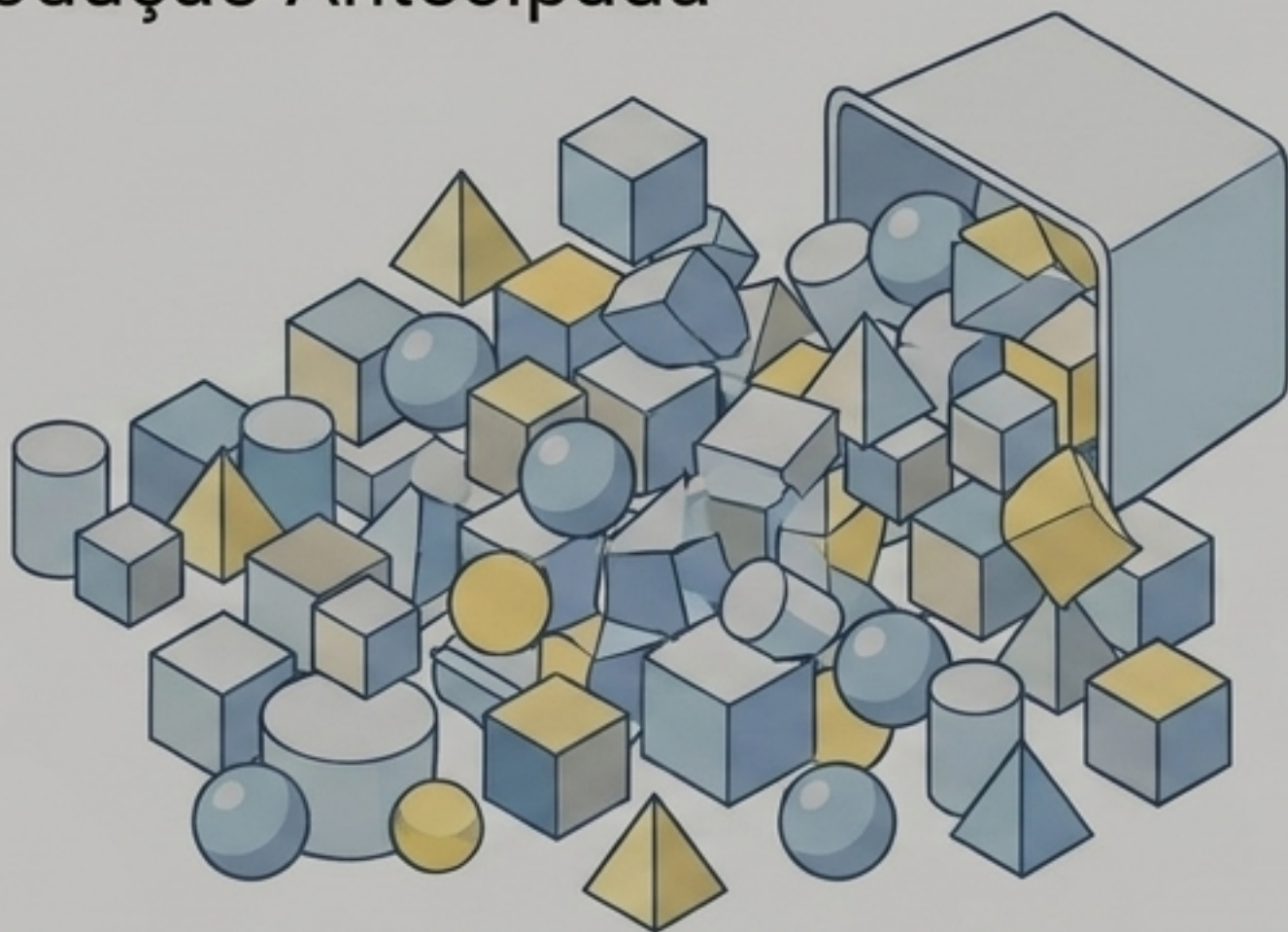
Retorna self. Um padrão comum quando a classe implementa ambos os protocolos.

Zona 3: O Motor

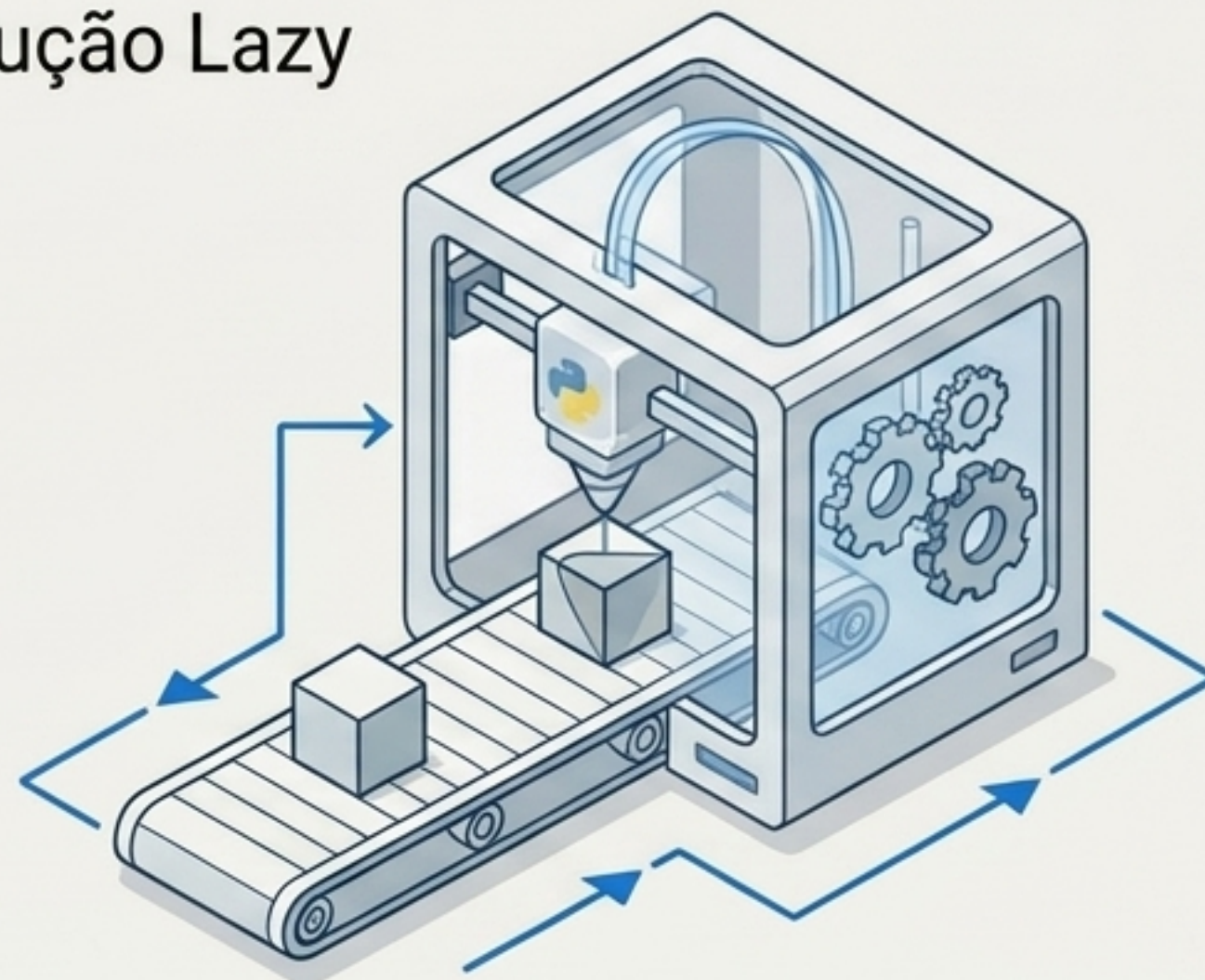
A lógica de negócio e a condição de parada.

Generators: Produção Sob Demanda

Produção Antecipada



Produção Lazy

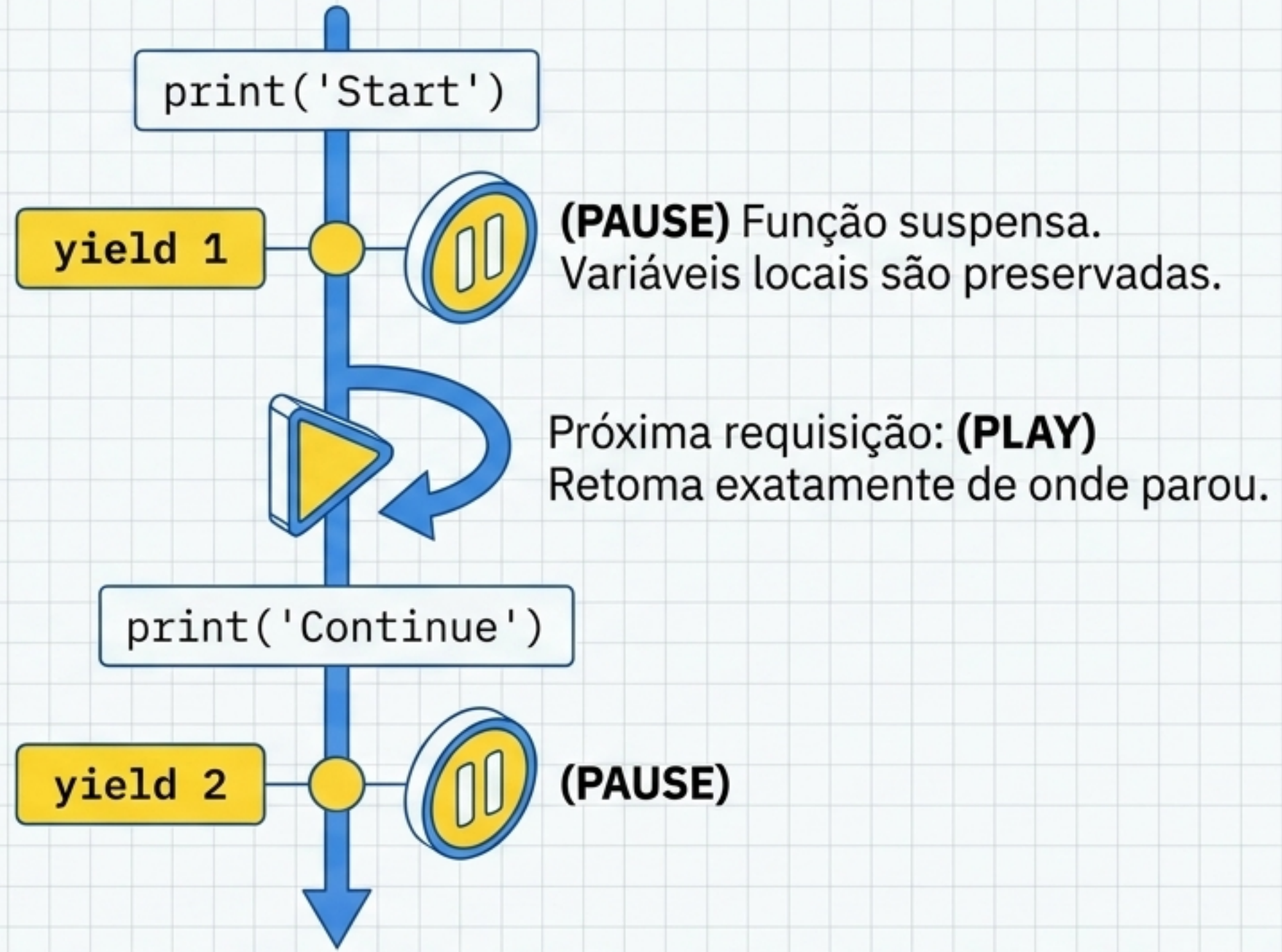


Lazy Evaluation: Dados são criados no exato momento em que são necessários.

Performance: Economia massiva de memória em datasets gigantes.

A Chave Mágica: A palavra reservada **yield** (transforma qualquer função comum em uma função geradora).

A Mágica do yield: Congelamento de Estado



Um return destrói o estado da função. Um yield apenas a coloca para dormir.

Matriz de Redução de Código: Class vs Generator

Iterador Tradicional

```
class Evens(object):  
    def __init__(self, limit):  
        self.limit = limit  
        self.val = 0  
    def __iter__(self): return self  
    def __next__(self):  
        if self.val > self.limit:  
            raise StopIteration  
        else:  
            return_val = self.val  
            self.val += 2  
            return return_val
```

VS

Gerador Moderno

```
def evens_up_to(limit):  
    value = 0  
    while value <= limit:  
        yield value  
        value += 2
```

O Python encapsula automaticamente os protocolos **`__iter__`** e **`__next__`** por baixo dos panos, limpando o código.

Controle Manual Fora do Loop

```
evens = evens_up_to(4)
```

```
print(next(evens)) # Saída: 0
```

```
print(next(evens)) # Saída: 2
```

```
print(next(evens)) # Saída: 4
```

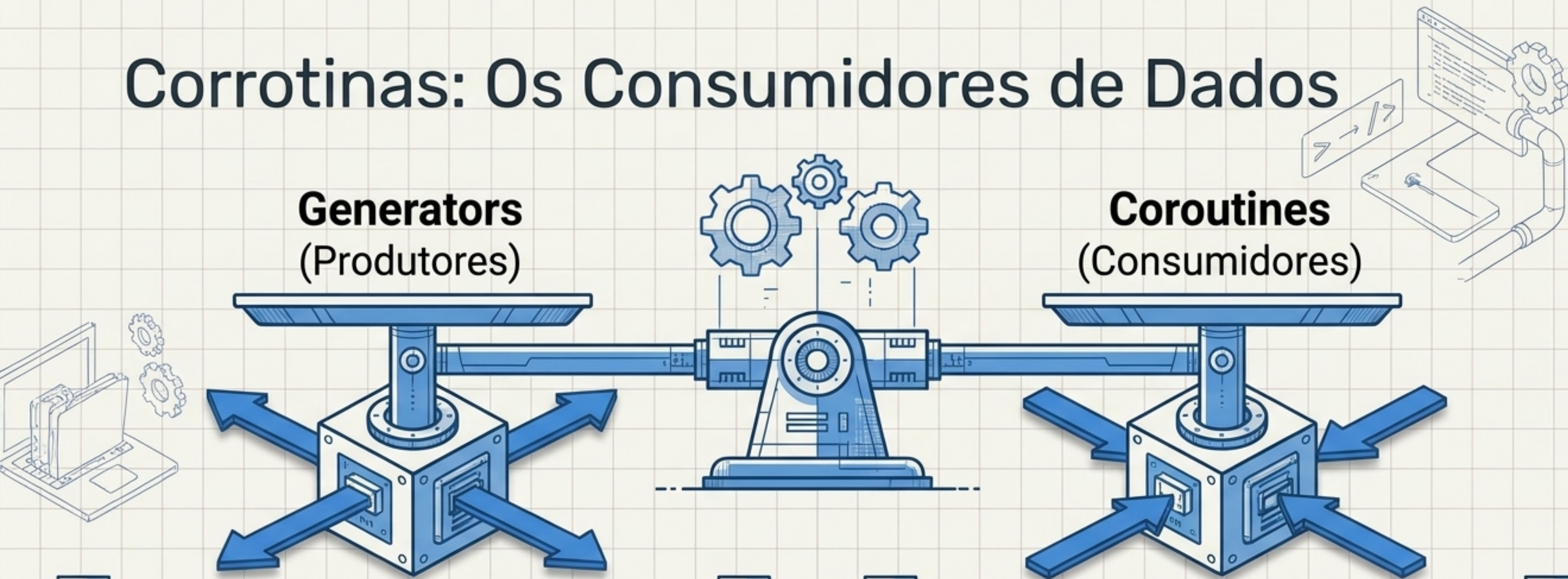


E se chamarmos `next(evens)` novamente?

O gerador dispara a exceção **StopIteration**.

Detalhe Adicional: Múltiplas chamadas (ex: `evens_up_to(4)` e `evens_up_to(6)`) criam objetos de estado completamente separados, possibilitando aninhamento seguro e instâncias independentes.

Corrotinas: Os Consumidores de Dados

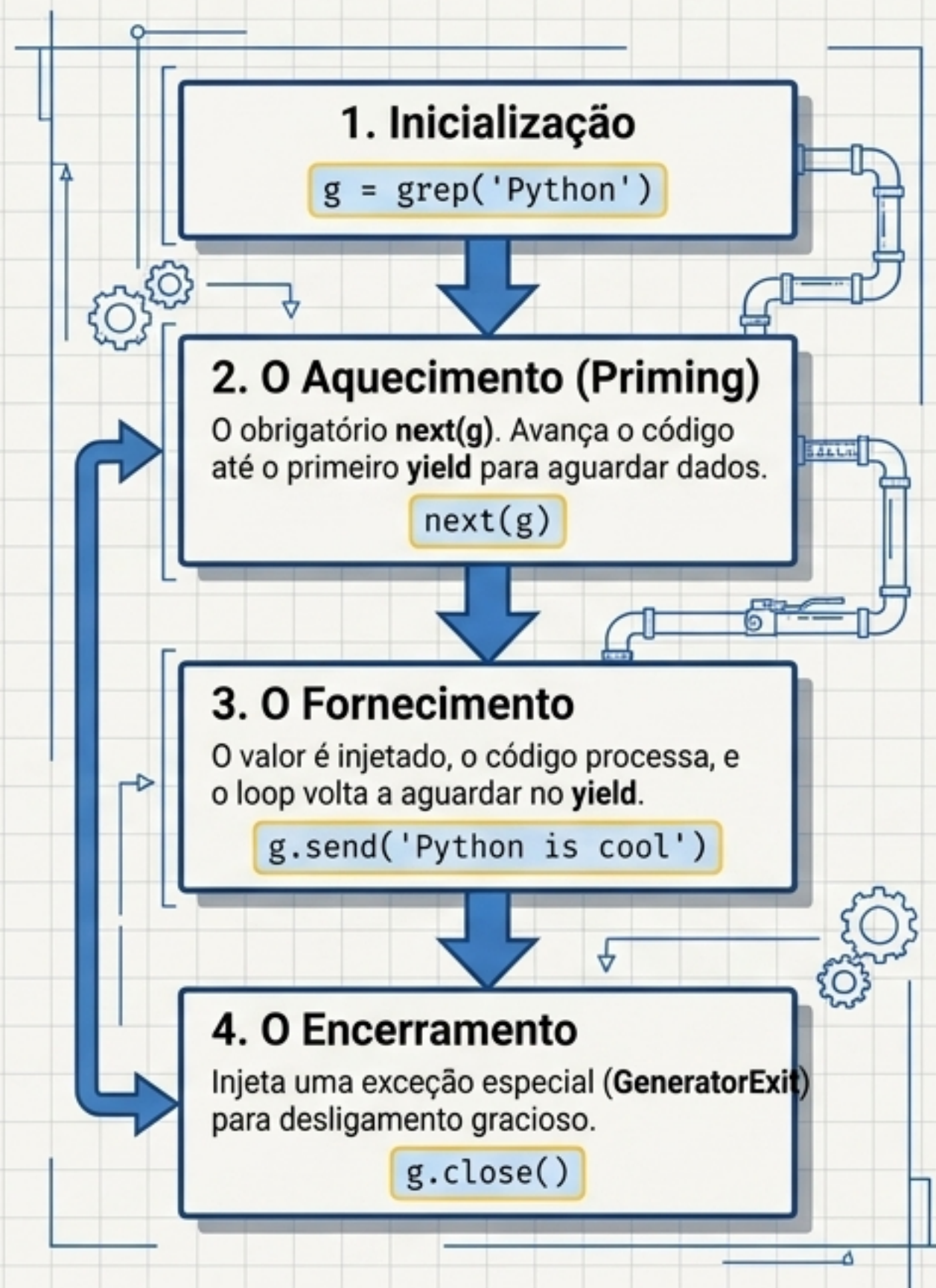


Corrotinas esperam passivamente que valores sejam fornecidos a elas.

A palavra **yield** muda de função: agora atua como uma atribuição de variável.

```
valor_recebido = (yield)
```


Anatomia e Ciclo de Vida de uma Corrotina



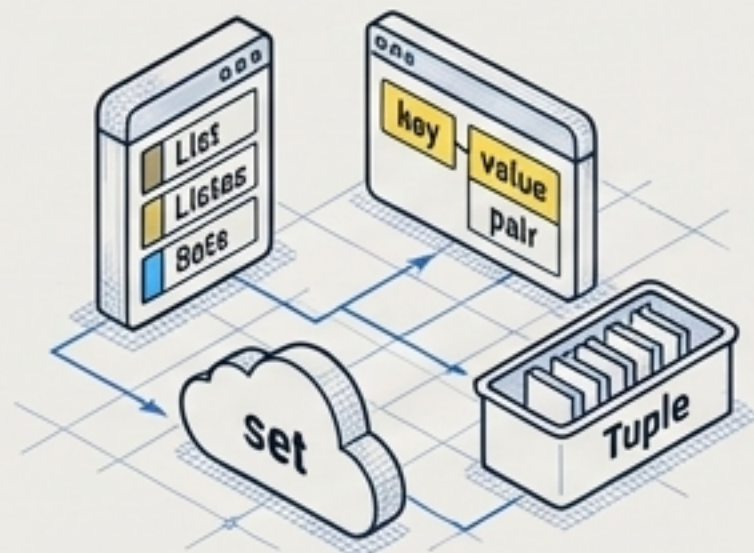
Matriz de Diagnóstico: Geradores vs. Corrotinas

Gerador	Corrotina
Papel Principal: Produtor de Dados	Papel Principal: Consumidor de Dados
Uso do yield: <code>yield valor</code> (expele o dado)	Uso do yield: <code>var = (yield)</code> (ingere o dado)
Método de Ação: Acionado primariamente por <code>next()</code>	Método de Ação: Acionado primariamente por <code>.send(dado)</code>

Resumo: O Mapa da Iteração

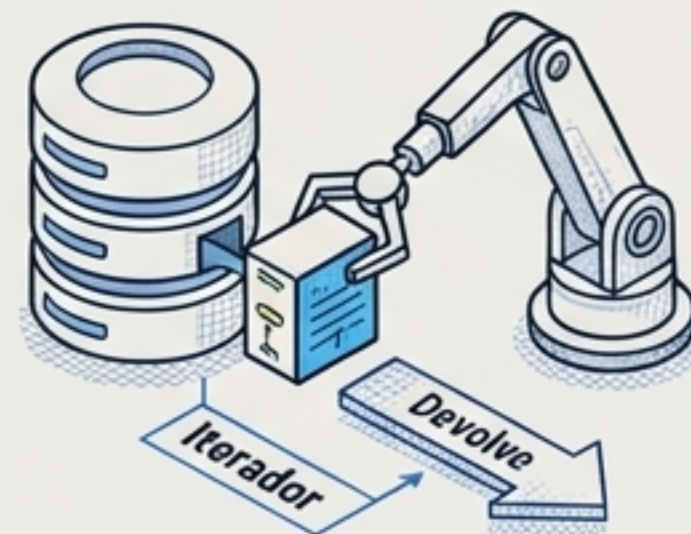
1. Coleções

Recipientes estáticos
(Tuplas, Listas, Sets,
Dicts).



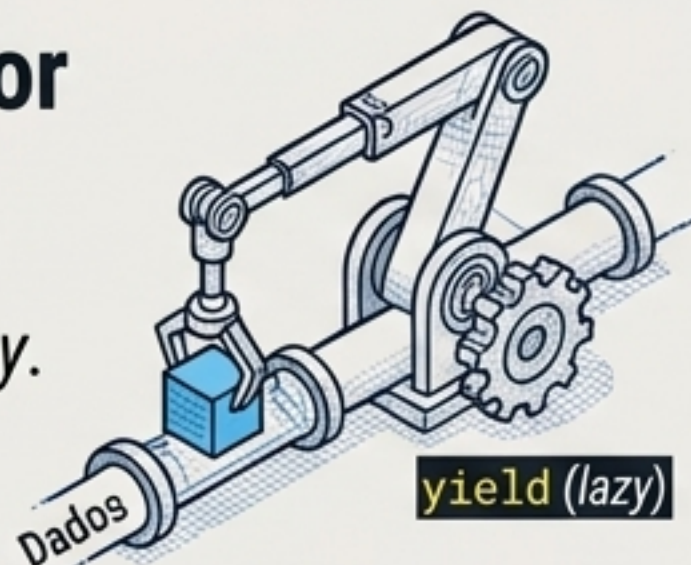
2. Iterable

O dono dos dados.
Implementa `__iter__()`.
Sempre devolve um
Iterador.



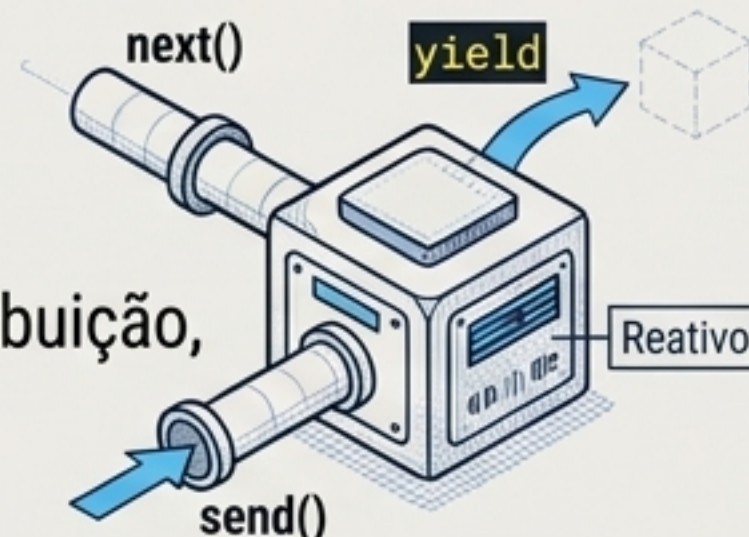
3. Iterator / Generator

O motor de busca.
Implementa `__next__()` /
usa **yield** para avaliação *lazy*.
Produz dados.



4. Corrotinas

Consumidores reativos.
Iniciam com `next()`,
aguardam **yield** como atribuição,
alimentados via `send()`.



A iteração não é apenas um loop mecânico; é a orquestração eficiente de dados e estado.